Class Project – Defense

Group 6 Hands-on Technical Report (Blue Team)

M143040024　陳品叡
M143140012　呂易鴻
M143140017　王承煜

2026-06-05

Abstract

This report is the defensive companion to our Group 6 attack experiment. Where the attack report reconstructed an end-to-end Cyber Kill Chain – reconnaissance, honeypot probing and IP-alias bypass, Flask/Jinja2 SSTI, fileless ICMP command-and-control, a TCP reverse shell, and DNS/ICMP data exfiltration – this document explains *how the Blue Team detects and responds to each of those behaviors*, what tools implement each control, and, in detail, *where the defense still fails*. The defense is organized as two enforcing layers plus a monitoring plane: (1) a **network layer** combining a fake-SSH honeypot with an `iptables` auto-blocker; (2) a **kernel layer** built on eBPF that hooks security-relevant syscalls and can terminate a process in kernel space *before* the syscall completes; and (3) a **SOC dashboard** that aggregates telemetry from both layers in real time. We describe the exact detection logic of each eBPF hook (the fileless-staging detections plus the `dup2`/`dup3` reverse-shell hook, all in `blue_ebpf_mdr_v2.py`), the cold-start `/proc` scanner, and the kernel-space kill mechanism. We then give an honest evaluation: behavior-based, source-IP-agnostic kernel detection is far stronger than IP-based blocking, but it has real blind spots – most importantly, covert channels built from *legitimate* syscalls (DNS/ICMP exfiltration) and persistence (`crontab`) survive the full defense stack. The central lesson is that defense-in-depth is a continuous process, not a finished product.

Contents

1 Introduction

1.1 Project context

The project implements a self-contained red–blue lab (target service, honeypot, eBPF detection, C2, and exfiltration) that walks the full Cyber Kill Chain and maps each step to MITRE ATT&CK. The attack report (Hands-on 2) documented the offensive path. **This report takes the defender's seat**: it describes the controls the Blue Team operates, the order in which they are brought online, the evidence each one produces, and the residual risk that remains after every control is running. The red team is referenced only where it is needed to explain *what* a given control is defending against.

1.2 Defensive methodology

We follow three principles.

1. **Defense in depth.** No single control is treated as a complete stop. Each layer is expected to fail against some class of attack; the point of layering is that the next layer catches what the previous one missed.
2. **Behavior over signature.** The kernel layer does not match file hashes or IP reputation; it matches *actions* (syscalls and their arguments). This is what lets it catch fileless malware that never touches disk and survives an attacker changing source IP.
3. **Evidence-driven, code-accurate.** Every claim about what a control does is taken from the tool source, not from the demo script. Where the demo narrative and the code disagree, we describe the code's actual behavior and flag the difference (see §5.7).

1.3 Objectives

1. Describe the Blue Team architecture and the role of every defensive component.
2. Document the detection-and-response *workflow*: how telemetry flows from sensors to the SOC, and how the operator drives the controls round by round.
3. Give a precise, accurate account of each eBPF hook and the kernel-space kill mechanism.
4. Critically evaluate the limitations – what gets through, why, and what would be required to close each gap.

1.4 Scope and ethics

All activity ran in an isolated lab with explicit authorization for education. The target is purpose-built for the course. The defensive tools (honeypot, network MDR, eBPF MDR, SOC dashboard) are general-purpose detection code; nothing here is intended for, or safe to run against, systems you do not own.

2 Threat Model and Defensive Frame

2.1 What we are defending against

From the defender's point of view, the relevant attacker behaviors – abstracted away from the specific exploit – are:

- **Probing / target selection** against exposed services.
- **Application-layer code execution** (SSTI) that turns a web request into arbitrary Python on the host.
- **Fileless staging**: code is created as an anonymous in-memory file descriptor (`memfd_create`) and executed via `/proc/<pid>/fd/<N>`, leaving no file on disk to scan.
- **Covert command-and-control**: control traffic hidden inside ICMP echo payloads (encrypted), or a plain TCP reverse shell.
- **Data exfiltration** over alternative protocols (Base32-over-DNS, hex-over-ICMP).
- **Persistence** via `crontab` that re-downloads the agent.

2.2 Defense-in-depth model

The controls form two enforcing layers under one monitoring plane (Figure 1).

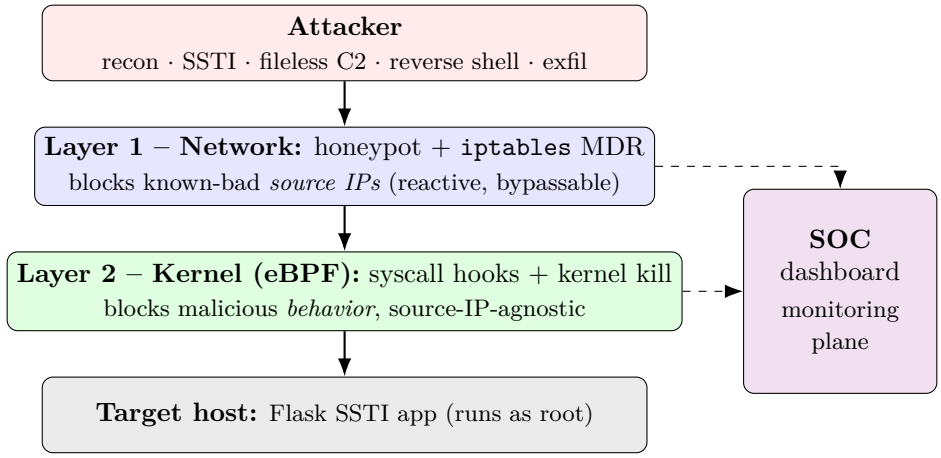

Figure 1: Defense-in-depth. Each layer is expected to fail against some class of attack; the next layer—and the SOC monitoring plane—watches the gap. Solid arrows = attack progression; dashed = telemetry to the SOC.

The ordering matters: Layer 1 is cheap, generates early signal, and buys time, but is trivially bypassed (§4.4); Layer 2 is the real enforcement point because it acts on behavior the attacker cannot avoid if they want code execution and a control channel.

2.3 Detection coverage (MITRE ATT&CK)

Table 1 maps each attacker technique to the control that detects it, the layer, and whether that control *enforces* (kills/blocks) or only *alerts*. The enforce-vs-alert column is important and is justified per-hook in §5.

The three "Gap" rows are not oversights; they are the honest boundary of this defense and are analyzed in §8.

Table 1: Detection-and-response coverage. "Enforce" = the control can stop the action; "Alert" = the control records it but does not stop it.

ATT&CK	Technique	Control / hook	Layer	Action
T1595	Active Scanning	Honeypot (`honeypot.py`)	Network	Alert
–	(response)	`iptables` MDR	Network	Enforce (block IP)
T1190	Exploit Public-Facing App	(not directly detected)	–	Gap
T1620	Reflective Code Loading	`sys_enter_memfd_create`	Kernel	Alert + taint
T1059	Exec from `/proc/fd`	`sys_enter_execve` (argv)	Kernel	Enforce (kill)
T1095	Non-App-Layer (ICMP)	`sys_enter_socket` (raw ICMP)	Kernel	Enforce if correlated
T1571	Non-Standard Port	`sys_enter_connect`	Kernel	Alert
T1059.006	Reverse-shell fd hijack	`sys_enter_dup2/dup3`	Kernel	Enforce (kill)
T1048.003	Exfil over DNS/ICMP	(not detected)	–	Gap
T1053.003	Persistence: Cron	(not detected)	–	Gap

3 Defensive Architecture and Environment

3.1 Machines and roles

Two native Ubuntu 24.04 machines are used. The **lab machine** hosts the target service *and* the entire Blue Team stack; the **attacker machine** runs the red-team tooling. Only the lab machine needs eBPF and root.

Machine	Role	Runs
Lab	Target + Blue Team	target app, honeypot, eBPF MDR, network MDR, SOC dashboard
Attacker	Red Team	recon, C2, reverse shell, exfil (out of scope here)

3.2 Defensive components

Table 2: Blue Team components and their function.

File	Layer	Function
`target/honeypot.py`	Network	Fake SSH on :2222; logs every connector IP to `trap.log`
`blue_team/blue_mdr_network.py`	Network	Watches `trap.log`; `iptables -j DROP` on new IPs
`blue_team/blue_ebpf_mdr_v2.py`	Kernel	eBPF MDR: memfd/execve/raw-ICMP staging detection + correlation, connect (suspect ports), dup2/dup3 reverse-shell
`blue_team/soc_dashboard.py`	Monitoring	Flask + SSE timeline over `trap.log` and `soc_events.jsonl`
`cleanup.sh`	Ops	Resets processes, `iptables`, IP aliases, logs, crontab

3.3 Telemetry and data flow

There are two log sinks, both at the project root, and one in-kernel buffer:

- `trap.log` – written by the honeypot, one line per connection, in the form `[ts] Attacker`

`IP: <ip> Port: <p> Data: <...>`. It is the contract between the honeypot and the network MDR.

- `soc_events.jsonl` – one JSON object per line, written by the eBPF MDR (kill/alert events) and by the network MDR (IP blocks).
- **perf ring buffer** – a lock-free, `mmap`-shared buffer the eBPF program writes and the Python userspace reads (`page_cnt=64`, i.e. 256 KB).

Figure 2 shows how the sinks are fed and consumed: three producers write two files, and the SOC dashboard tails both.

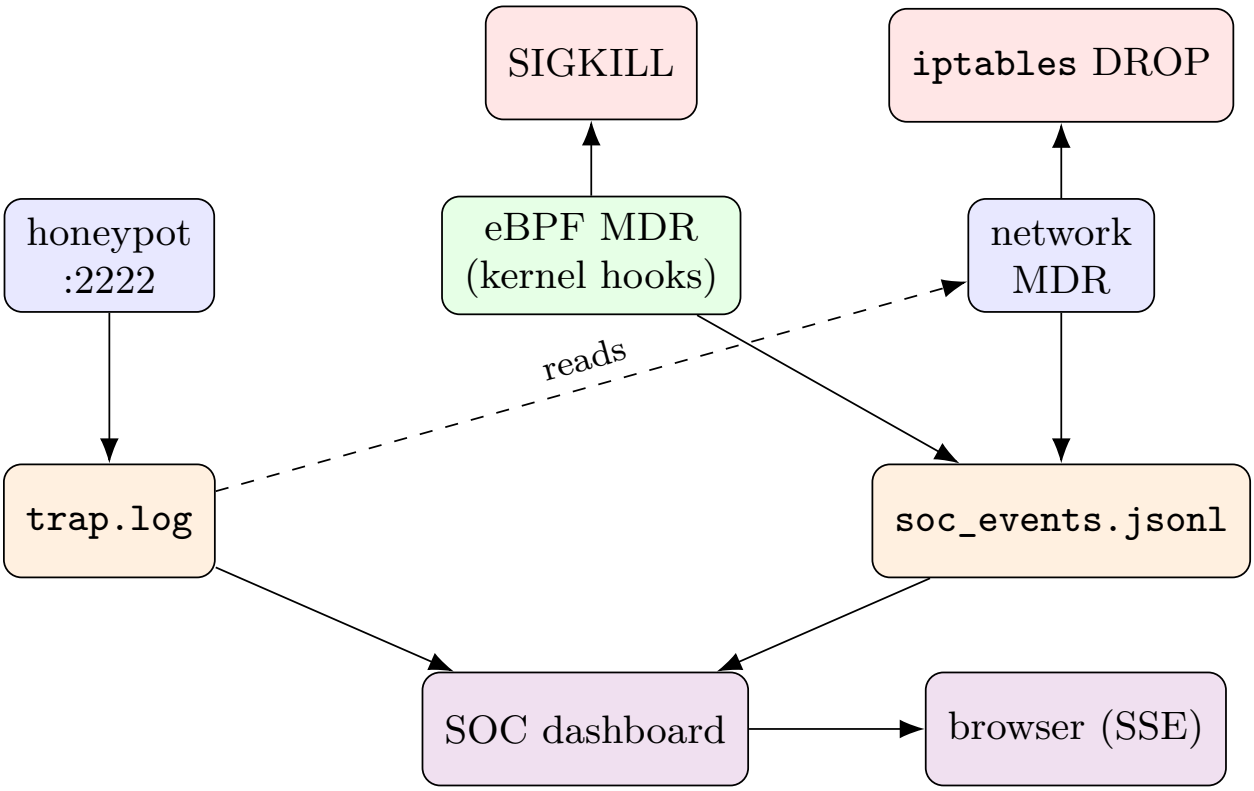

Figure 2: Telemetry flow. Producers (honeypot, kernel eBPF MDR, network MDR) write two sinks (`trap.log`, `soc_events.jsonl`); the SOC dashboard tails both and streams them to the browser over SSE. Enforcement (SIGKILL, `iptables` DROP) is a side effect, not telemetry.

4 Layer 1 – Network-Layer Defense

4.1 Deception: the fake-SSH honeypot

The honeypot listens on TCP 2222 and, on every connection, sends a banner that is byte-for-byte a real OpenSSH version string:

```
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.4
```

To `nmap -sV` this is indistinguishable from a genuine SSH service, so a scanner is drawn to it. The server then reads whatever the client sends (its SSH client hello), returns a fake "Permission

denied" rejection, and records the connection.

The defensive value is **a near-zero false-positive signal**: nothing legitimate has any reason to connect to this port. Any hit is, by construction, suspicious. Each hit is appended to `trap.log` with the source IP, port, and the first 100 bytes the client sent (useful for later triage). This maps to detecting reconnaissance (T1595) by *inviting* it into a monitored trap.

4.2 Automated response: the network MDR

`blue_mdr_network.py` is a small daemon that tails `trap.log`. On startup it seeks to the current end of the file (so stale entries from a previous run are ignored), then polls (default 1 s) for new lines. For each new, valid, not-already-blocked IPv4 address it runs:

```
iptables -I INPUT 1 -s <attacker_ip> -j DROP
```

Inserting at position 1 makes the DROP the highest-priority INPUT rule, so the attacker is cut off from *all* ports on the host – not just 2222 – in under a second. The block, the IP, and the action are also written to `soc_events.jsonl` (`source: NETWORK_MDR, event: IP_BLOCKED`). With `--cleanup`, every rule the daemon added is removed on exit, so the lab returns to a clean state.

4.3 Layer-1 defensive process

1. Operator starts the honeypot and the network MDR (with `--cleanup`) from the same repo directory, so both resolve the *same* `trap.log` path.
2. Attacker probes :2222 (deliberately or via a scan).
3. Honeypot replies with a real-looking banner and logs the IP.
4. MDR sees the new line within one poll interval and inserts a DROP.
5. Attacker's original IP can no longer reach the host on any port.

4.4 Strengths and limitations of Layer 1

The honeypot is excellent *signal* (high confidence, low noise) and the auto-block is fast, but the control is fundamentally **reactive and IP-scoped**, which is its undoing:

- It only blocks IPs that have *already* touched the trap. An attacker who never connects to :2222 is never blocked by this layer.
- Blocking is by source IP, and **source IP is attacker-controlled**. In the exercise, the red team simply adds an alias IP (`ip_switch.sh`) and continues from a fresh identity – the real target on :9999 is reachable again immediately. NAT, proxies, and IP rotation generalize this bypass.

The conclusion the lab draws is the right one: IP-based blocking is a useful *first* layer that buys time and produces telemetry, not a standalone preventive control. That is precisely why a second, behavior-based layer is required.

5 Layer 2 – Kernel-Layer Defense (eBPF MDR)

5.1 Why eBPF

eBPF lets us run a small, verified program inside the kernel, attached to syscall *tracepoints*. This gives the defender four properties that userspace agents cannot match:

- **Source-IP-agnostic.** It acts on what a process *does*, so the Layer-1 IP-alias bypass is irrelevant here.
- **Fileless-visible.** It sees `memfd_create` and `execve` arguments directly, so in-memory malware that never writes a file is still observable.
- **Pre-syscall enforcement.** Hooks fire at *syscall entry* (`sys_enter_*`), before the kernel performs the action. A kill here breaks the chain before the dangerous operation completes.
- **Low overhead.** Hash-map lookups are $O(1)$ and the program is JIT-compiled to native code by the kernel.

5.2 The eBPF pipeline

BCC compiles the embedded C to eBPF bytecode with Clang/LLVM at runtime; the kernel *verifier* proves it is safe (bounded loops, no wild pointers); the JIT converts it to native x86-64; it is attached to tracepoints; and it communicates with userspace through the perf ring buffer and shared hash maps. Two design details follow directly from the verifier: every string scan is a *bounded* loop (`for (i=6; i<20; i++)`), and all userspace reads go through `bpf_probe_read_user[_str]` with explicit sizes (Figure 3).

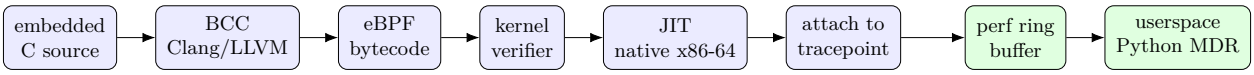

Figure 3: The eBPF pipeline. Blue = load time (compile, verify, JIT, attach); green = run time (events surface to userspace through the perf ring buffer). The verifier is the reason every string scan is a bounded loop.

5.3 Fileless-staging detections

The eBPF MDR (`blue_ebpf_mdr_v2.py`) installs seven probes in total; the first four target the fileless-staging chain.

Hook 0 – fork tracking (`sched_process_fork`). When a process that previously called `memfd_create` forks, the child gets a new PID that the correlation logic would otherwise miss. This hook copies the parent's entry in the `memfd_pids` map to the child, so the "memfd taint" propagates across `fork+execve`. Without it, the correlation in Hook 3 would be defeated by a single fork.

Hook 1 – `memfd_create` (fileless staging). On entry, the hook records `(pid, timestamp)` in `memfd_pids` and captures the anonymous file name. **It is alert-only**: it never kills (the kill placeholder is empty in both modes). Its job is to *taint* the process for later correlation, not to enforce – `memfd_create` has a few legitimate users and killing on it alone would be too aggressive.

Hook 2 – `execve` from `/proc/<pid>/fd/<N>`. This is the actual enforcement point of the fileless chain. The hook inspects two things:

- the `execve` *filename*; if it is itself `/proc/.../fd/...` (used legitimately by systemd's `sd-executor`) it is **alert-only**;
- `argv[1]`; if *that* points at `/proc/.../fd/...` (the pattern `python3 /proc/<pid>/fd/<N>` used by the fileless loader) it is **killed**.

This filename-vs-argv distinction is a deliberate false-positive control: it separates a legitimate systemd pattern from the red-team pattern and only enforces on the latter.

Hook 3 – raw ICMP socket + correlation. The socket hook filters precisely for `socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`. A standalone raw ICMP socket (e.g. `ping`) is **alert-only**. But if the calling PID *or its parent* is in `memfd_pids`, we have a high-confidence correlation – *fileless staging plus a covert ICMP channel* – which is extremely unlikely to be benign, and it is **killed**. This multi-indicator correlation is the staging detector's strongest idea: it raises confidence enough to justify an automated kill.

5.4 Cold-start `/proc` scanner

eBPF tracepoints only fire on *new* syscalls. If the agent was deployed *before* the Blue Team started monitoring, the hooks never see it. The tool compensates with a one-shot `/proc` walk at startup that finds already-running fileless processes two ways:

1. `/proc/<pid>/exe` resolving to `memfd:*` (direct memfd exec);
2. a `python*` process whose `argv` references a live `/proc/<pid>/fd/<N>` that in turn points at a `memfd:` target (the Python fileless-loader pattern).

With `--kill`, matches are terminated with a userspace `SIGKILL`. This is a *compensating* control: it narrows, but does not eliminate, the temporal gap (§8.3).

5.5 Reverse-shell detection

A TCP reverse shell uses only ordinary syscalls (`socket(SOCK_STREAM)` → `connect` → `dup2` → `pty.spawn`), so none of the four staging hooks above fire on it. The remaining three hooks exist to close that gap.

Hook 4 – `connect` to a suspect port. The hook reads the first 8 bytes of the `sockaddr` the program passed to `connect`, checks the family is `AF_INET`, byte-swaps the port to host order, and looks it up in a `suspect_ports` hash map populated from Python at startup (default {4444, 4445, 5555, 1234, 1337}). On a hit it records the destination IP and port. **It is alert-only** (§5.7).

Hook 5/6 – `dup2` / `dup3` fd-hijack. A reverse shell redirects all three standard descriptors to the socket: `dup2(s,0)`, `dup2(s,1)`, `dup2(s,2)`. The hook keeps a per-PID bitmask; each redirect of fd 0/1/2 sets a bit. When the mask reaches `0x07` (all three), a reverse shell is confirmed and the process is **killed**. `dup3` is hooked as well because Python's `os.dup2(..., inheritable=False)` actually calls `dup3`; without it an attacker bypasses Hook 5 with one keyword argument. Crucially, this signal fires *regardless of port*: even a reverse shell on 80/443 (to blend in) is caught the moment it hijacks the three fds. Figure 4 shows this as a per-PID bitmask state machine.

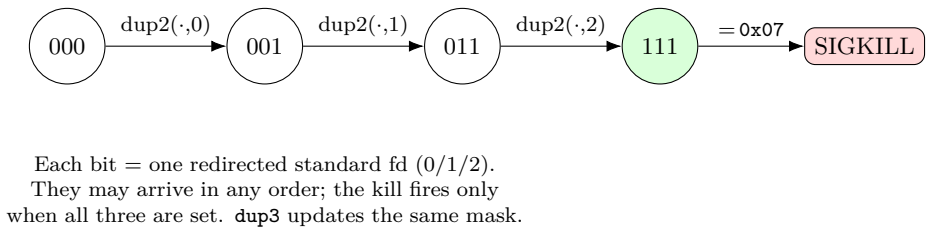

Figure 4: Reverse-shell detection as a per-PID bitmask (`sys_enter_dup2/dup3`). Reaching `0x07` (stdin+stdout+stderr all redirected to the socket) confirms a shell hijack on *any* port.

5.6 Kernel-space active response

The kill is `bpf_send_signal(9)` (SIGKILL) executed inside the hook at syscall entry. Because the hook runs before the kernel performs the syscall, the target is killed with no userspace race and no window for the action to complete. The tool injects this by string-replacing per-hook placeholders (`__KILL_*__`) at load time – one eBPF source, two behaviors (monitor vs. `--kill`) – which avoids maintaining two copies of the C.

5.7 Which hooks actually enforce

This is the single most important nuance of the kernel layer, and it differs from the demo script. Reading the load-time injection in `blue_ebpf_mdr_v2.py`, even with `--kill`:

Hook	`--kill` behavior	Why
`memfd_create`	alert only	low confidence alone; used for taint
`execve` (filename)	alert only	legitimate systemd pattern
`execve` (argv)	**kill**	fileless-loader pattern, high confidence
`socket` raw ICMP	**kill if correlated**	memfd+ICMP is high confidence
`connect` suspect port	alert only	port alone is high false-positive
`dup2/dup3` mask==0x07	**kill**	three-fd hijack is unambiguous

Two consequences worth stating plainly:

- For the **fileless ICMP C2 chain**, the break point is the `execve(argv)` kill (with the ICMP correlation as a backstop) – *not* `memfd_create`. The demo script labels the `MEMFD_CREATE` line as `KILLED`; the code does not kill there.
- For the **reverse shell**, `connect` only *alerts*; the enforcing signal is `dup2/dup3` reaching `0x07`. So the TCP connection does briefly succeed, and the kill lands during fd setup, before an interactive shell exists. The demo script attributes the kill to `SUSPECT_CONNECT`; the code enforces at the fd-hijack.

We read this as a sound design choice – enforce on the highest-confidence signal, alert on the noisy ones – but it should be documented, because it changes *when* and *why* the attack actually stops.

5.8 Safety controls

A `whitelist` hash map holds PIDs that are never killed (the MDR always whitelists its own PID; more can be added with `--whitelist`). The tool also runs monitor-only by default; `--kill` must be explicit. These are the guardrails against the active-response risk discussed in §8.5.

6 Detection-and-Response Workflow

6.1 The SOC dashboard

`soc_dashboard.py` is a Flask app that tails both `trap.log` and `soc_events.jsonl`, normalizes each into a common event shape, and streams them to a dark-theme web console over Server-Sent Events. It keeps running counters (total events, blocked IPs, process kills, critical alerts) and renders a live, severity-coded timeline. It is **a monitoring plane only** – it does not enforce – but it is what turns three separate console tools into a single operator view, and it exposes a `POST /api/event` endpoint so other tools can push events directly.

6.2 Operational startup (runbook)

The defender brings controls online in this order (all from one repo directory, all through the project venv):

```
# Lab T1 -- target + deception
sudo .venv/bin/python3 target/target_app.py
sudo .venv/bin/python3 target/honeypot.py

# Lab T2 -- Layer 1 network MDR (auto-clean its rules on exit)
sudo .venv/bin/python3 blue_team/blue_mdr_network.py --cleanup

# Lab T2 -- Layer 2 kernel MDR
sudo .venv/bin/python3 blue_team/blue_ebpf_mdr_v2.py --kill

# Lab -- monitoring plane
python3 blue_team/soc_dashboard.py     # browse http://<lab>:8080
```

6.3 Round-by-round defensive narrative

The exercise is most instructive read as a defender's timeline. The following is the *defense* viewpoint of each round; attacker steps are summarized only as triggers. Figure 5 summarizes the outcome of each round.

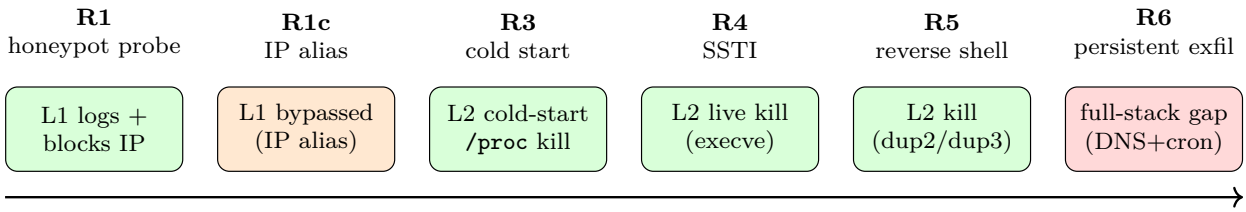

Figure 5: Attack–defense rounds from the defender's side. Green = attack stopped; amber = a layer is bypassed (hand-off to the next layer); red = a gap nothing in the stack closes. Time flows left to right.

1. **Recon / honeypot trap (R1–1b).** Attacker hits :2222. Honeypot logs the IP; network MDR inserts an `iptables` DROP in <1 s; attacker's original IP loses all access. *Layer 1 works.*
2. **IP-alias bypass (R1c).** Attacker adds an alias IP and reaches :9999 again. *Layer 1 is bypassed – exactly as expected; hand off to Layer 2.*

3. **Cold-start cleanup (R3).** Blue Team starts the eBPF MDR *after* the agent is already running. The `/proc` scanner finds the in-memory `python3 /proc/<pid>/fd/<N>` agent and kills it; the attacker's C2 goes silent. *Compensating control works.*

4. **Real-time interception (R4).** Attacker re-runs the SSTI. The chain is now killed live at the `execve(argv)` hook (memfd is tainted/alerted first), so no agent ever beacons. *Layer 2 hot path works.*

5. **Reverse shell (R5).** Attacker switches to a TCP reverse shell (no memfd, no raw ICMP), so the four staging hooks stay silent by design. The same tool still catches it: `connect` raises an alert, and the process is killed the moment `dup2/dup3` redirects all three standard fds to the socket (`0x07`). *Caught on the highest-confidence signal, no version change required.*

6. **Persistent DNS exfiltration (R6).** A `crontab` planted earlier re-downloads an exfil agent that ships data over DNS. It uses only `socket(SOCK_DGRAM)` + `sendto` – all legitimate syscalls – so the eBPF MDR raises *no alert*, and the persistence survived every kill. *Full-stack gap (§8).*

6.4 Reset between runs

`cleanup.sh` returns the lab to a known-good state: it kills leftover processes, flushes the `iptables` rules the MDR added, removes IP aliases, truncates `trap.log`/`soc_events.jsonl`, clears `loot/`, and removes the persistence `crontab`. `--dry` previews without acting. Operationally this matters: a stale `trap.log` would make the MDR appear unresponsive, and a surviving crontab would re-infect the next run.

7 Evaluation and Results

7.1 What the defense demonstrably stops

- **Known-IP reconnaissance/return** – blocked at Layer 1 (sub-second), with the explicit caveat that it is bypassable.
- **Fileless ICMP C2** – stopped at Layer 2 both *cold* (`/proc` scan kills an already-running agent) and *hot* (`execve(argv)` kill, ICMP-correlation backstop) so no beacon is established.
- **TCP reverse shell** – stopped at the `dup2/dup3` fd-hijack, independent of the C2 port.

7.2 Response characteristics

The kernel kill is *pre-completion*: the SIGKILL is issued inside the `sys_enter` hook, so there is no userspace polling race for the *enforcing* path. (The *alerting* path – console line and JSONL write – does run in userspace after the perf-buffer read, and can lag or drop under load; see §8.8.) The network block is bounded by the MDR poll interval (default 1 s).

7.3 What it does *not* stop

Three techniques traverse the entire stack untouched: SSTI itself (the root cause), DNS/ICMP exfiltration (legitimate syscalls), and `crontab` persistence. These define the next section.

8 Limitations and Open Problems

This is the core of the defensive analysis: a control you cannot describe the failure modes of is a control you do not understand.

8.1 Network layer: IP blocking is reactive and bypassable

Already shown in §4.4: the block is keyed on a value the attacker controls. IP aliasing defeated it in seconds; NAT, proxy chains, cloud egress, and shared-IP environments make it worse (a block can even harm innocent co-tenants). It is signal and friction, not prevention.

8.2 Kernel layer: legitimate-syscall covert channels are invisible

The eBPF layer matches *specific dangerous syscalls and arguments*. The DNS exfiltration agent uses `socket(AF_INET, SOCK_DGRAM)` and `sendto` to port 53 – ordinary, ubiquitous syscalls with no distinguishing argument the current hooks inspect. There is:

- no `memfd_create`, so no taint;
- no `SOCK_RAW`, so the socket hook's filter (`type==SOCK_RAW`) never matches;
- no `dup2/dup3` fd-hijack;
- no `connect` to a suspect port (DNS uses 53, and uses `sendto`, not `connect`).

The data is also Base32/Base64-encoded inside DNS subdomains, so even content inspection would need DNS-aware parsing. **This is the most important gap**: behavior-based detection that enumerates "bad" syscalls is blind to malice expressed entirely through "good" ones. Catching it needs a different sensor – egress/DNS analytics, not syscall hooks.

8.3 Temporal gap: hooks only see new syscalls

Tracepoints fire going forward. Anything that happened before the MDR started is invisible to the hot path. The `/proc` cold-start scan (§5.4) narrows this, but it is a single snapshot, it only recognizes the two specific memfd patterns it is coded for, and a process that has already finished its sensitive syscalls (e.g. already exfiltrated) cannot be "un-run." Continuous re-scanning and earlier deployment are partial answers; the gap cannot be fully closed by a forward-looking sensor.

8.4 False positives and whitelist tuning

Every enforcing hook has a benign analogue:

- `memfd_create` is used by Chrome, systemd, and others – which is exactly why it is alert-only here, but it still adds noise.
- `execve` from `/proc/fd` is used by systemd `sd-executor`; the tool only avoids killing it because of the filename-vs-argv split, a heuristic, not a guarantee.
- `dup2(fd,0/1/2)` is normal for any shell or daemon doing I/O redirection. The `0x07`-all-three requirement reduces this, but a legitimate program that redirects all three standard fds to one descriptor would be killed.
- the `suspect_ports` list is *static*; it both misses C2 on unlisted ports and would false-positive on a legitimate service that happens to use one (a reason the connect hook is alert-only).

The whitelist is per-PID and manual, which does not scale: PIDs are recycled, and you cannot whitelist a process that does not exist yet.

8.5 Active-response risk

`bpf_send_signal(SIGKILL)` on a misclassified process is itself a denial of service – the detector becomes the outage. There is no quarantine, throttle, or "confirm" step: a single false positive on a critical daemon is fatal to it. This is why monitor-only is the default and why the enforcing hooks are restricted to the highest-confidence signals; but the residual risk is real and grows with the breadth of hooks.

8.6 Brittleness of argument matching

The detections are precise, which makes them evadable:

- The `execve` check matches the literal prefix `/proc/` and scans bytes 6–19 for `/fd/`. Symlinks, bind mounts, a longer PID pushing `/fd/` past byte 19, or invoking the interpreter differently can dodge the pattern.
- The `argv[1]` read uses a fixed pointer offset (`argv + 8`); it inspects only the second argument.
- The reverse-shell signal requires *exactly* fd 0,1,2. An attacker who multiplexes I/O differently, or redirects only one fd and uses the socket directly, never reaches `0x07`.

Bounded-loop and fixed-offset parsing are forced by the verifier, but they make the matcher a fixed target an adversary can study and step around.

8.7 Persistence is not addressed

The planted `crontab` re-downloads and re-runs the agent on a timer and survived every kill in the exercise, because killing a process does not remove the mechanism that launches it. No control here watches `cron`, `systemd` units, `.bashrc`, or other autostart surfaces. Eradication, not just termination, is missing.

8.8 Monitoring-plane and operational limits

- **Userspace alert path can drop.** The perf buffer is 256 KB; under a burst, events can be lost before Python reads them. The enforcing kill is unaffected (it is in-kernel), but the *record* of it may not reach the SOC.
- **No durable SIEM.** `soc_events.jsonl` and the SSE timeline are flat files and an in-memory list (capped at 1000); there is no retention, search, alerting, or cross-host correlation.
- **Per-host, root, BCC-dependent.** eBPF needs root, kernel headers, and a runtime Clang/LLVM compile (BCC). This is heavy to deploy at scale and fragile across kernel versions; a CO-RE/libbpf build would be far more portable.
- **Alert fatigue.** Alert-only hooks (memfd, standalone ICMP, suspect connect, systemd-pattern execve) generate volume that an operator must triage; without scoring this erodes attention.
- **Encrypted payloads.** C2 content is AES-encrypted, so the defense can only reason about *behavior*, never content – reinforcing that signature/content inspection is not an option here.

8.9 Root cause is untouched

The SSTI vulnerability (T1190) and the fact that the target runs as root are never fixed by any of these controls – they detect/limit *consequences*, not the entry point or the blast radius. A defense that stops the agent but leaves the injectable endpoint and root execution in place is winning rounds, not the game.

9 Hardening Recommendations and Future Work

Mapped to the gaps above:

1. **Close the exfiltration gap (§8.2).** Add an egress sensor: DNS query-rate and subdomain-entropy/length analytics, allow-listed resolvers, and monitoring of DNS-over-HTTPS. Detect ICMP payload volume/entropy anomalies. This is a different sensor class than syscall hooks and is the highest-value addition.
2. **Add eradication for persistence (§8.7).** Monitor `cron`, `systemd` units, and shell rc files; on a confirmed compromise, remove the launcher, not just the process.
3. **Move from single-signal kills to scoring (§8.4, §8.5).** Correlate `connect`+`dup2`+new-socket-fd into a confidence score; reserve auto-kill for high scores and quarantine/alert otherwise. Make the suspect-port list dynamic.
4. **Harden the sensor (§8.6, §8.8).** Normalize paths before matching; widen the argv scan; switch perf to the BPF ring buffer; rebuild on CO-RE/libbpf for portability and lower deploy cost.
5. **Build a real monitoring plane.** Ship events to a durable SIEM with retention, search, alerting, and multi-host correlation; attach response playbooks to alert classes.
6. **Fix the root cause (§8.9).** Patch the SSTI (no `render_template_string` on user input), drop the target's privileges from root, and add a WAF/input-validation layer so Layer 2 is a backstop, not the only thing standing between a web request and kernel-level code execution.

10 Conclusion

The Blue Team stack shows what layered, behavior-based defense buys you and where it stops. Layer 1 (honeypot + `iptables`) produces clean, early signal and sub-second blocking, but is reactive and trivially bypassed by an IP change. Layer 2 (eBPF) is the real enforcement: source-IP-agnostic, fileless-visible, and able to kill a process in kernel space before a dangerous syscall completes – it stopped both the fileless ICMP C2 (cold and hot) and the TCP reverse shell at the `dup2/dup3` fd-hijack. Yet the same design that makes it strong – enumerating specific dangerous syscalls – makes it blind to malice built from legitimate syscalls: DNS/ICMP exfiltration and a `crontab` persistence mechanism survived the entire stack. The honest takeaway is the project's thesis stated from the defender's side: defense-in-depth is a continuous process. Each layer is expected to fail against some class of attack; security comes from making sure the next layer, and the next sensor, is already watching the gap.

A Blue Team Commands

```
# --- environment (once) ---
bash setup_env.sh
source .venv/bin/activate

# --- Layer 1: deception + network MDR ---
sudo .venv/bin/python3 target/honeypot.py
sudo .venv/bin/python3 blue_team/blue_mdr_network.py --cleanup

# --- Layer 2: kernel MDR ---
sudo .venv/bin/python3 blue_team/blue_ebpf_mdr_v2.py --kill \
     --suspect-ports 4444,4445,5555,1234,1337 \
     --whitelist <pid1>,<pid2>

# --- monitoring plane ---
python3 blue_team/soc_dashboard.py --port 8080

# --- reset between runs ---
sudo bash cleanup.sh          # add --dry to preview
```

B eBPF hook reference

Tracepoint	Purpose
`sched/sched_process_fork`	propagate memfd taint to children
`syscalls/sys_enter_memfd_create`	fileless staging (alert + taint)
`syscalls/sys_enter_execve`	exec from `/proc/fd` (kill on argv)
`syscalls/sys_enter_socket`	raw ICMP socket (kill if correlated)
`syscalls/sys_enter_connect`	connect to suspect port (alert)
`syscalls/sys_enter_dup2`	reverse-shell fd hijack (kill at 0x07)
`syscalls/sys_enter_dup3`	dup3 variant of the above (kill at 0x07)

C SOC event schema (`soc_events.jsonl`)

```
{
  "ts":       "2026-06-05 14:30:01",   // event time
  "source":   "EBPF_v2",                // EBPF_v2 | NETWORK_MDR | HONEYPOT
  "event":    "REVERSE_SHELL",          // event label
  "severity": "CRITICAL",               // CRITICAL | HIGH | MEDIUM | INFO
  "ip":       "",                        // source/target IP if applicable
  "comm":     "python3",                // process name
  "action":   "KILLED",                 // KILLED | BLOCKED | ALERT | LOGGED
  "detail":   "PID:34567 PPID:34500 ..."// free-form context
}
```